

Afondamento nas Competencias Profesionais

Práctica 1.5

CI/CD - Coolify y Ngrok

Daniel Calvar Cruz

2º CS DAM

22 de enero de 2026

Índice

1. Introducción	3
2. Coolify, Ngrok	4
3. Prueba con API test simple	6
4. Prueba con API/BBDD	12
5. Abrir tunel fuera de la red	25
6. Auto-desplegar con webhook de Coolify	26
7. Info	30

1 Introducción

[Volver](#)

- ENUNCIADO -

- ENUNCIADO -

Requisitos previos:

- Creación de VM:
 - SO recomendado: Ubuntu.

2 Coolify, Ngrok

Volver

Instalación de Coolify y Ngrok:

<https://coolify.io/>

<https://ngrok.com/>

Tan solo hay que seguir las instrucciones en pantalla. Importante en Ngrok, crear cuenta y añadir Authtoken.

Self-hosted Installation

If you like taking control and managing everything yourself, self-hosting Coolify is the way to go.

It's completely free (apart from your server costs) and gives you full control over your setup.

⚡ Quick Installation (recommended):

```
curl -fsSL https://cdn.coollabs.io/coolify/install.sh | sudo bash
```

sh

Run this script in your terminal, and Coolify will be installed automatically. For more details, including firewall configuration and prerequisites, check out the guide below.

Note for Ubuntu Users:

The automatic installation script only works with Ubuntu LTS versions (20.04, 22.04, 24.04). If you're using a non-LTS version (e.g., 24.10), please use the [Manual Installation](#) method below.

Figura 1: Instalar Coolify

1

Connect

Get an endpoint online.



Agent

Linux

Choose another platform

Installation

[Apt](#) [Download](#) [Snap](#)

Install ngrok via Apt with the following command:

```
curl -sSL https://ngrok-agent.s3.amazonaws.com/ngrok.asc \
| sudo tee /etc/apt/trusted.gpg.d/ngrok.asc >/dev/null \
&& echo "deb https://ngrok-agent.s3.amazonaws.com bookworm main" \
| sudo tee /etc/apt/sources.list.d/ngrok.list \
&& sudo apt update \
&& sudo apt install ngrok
```

Run the following command to add your authtoken to the default **ngrok.yml** configuration file.

```
ngrok config add-authtoken 36kqwDWy2W0EMplhxC15ba5wNff_7738oSN3TRkft4q7bEPz9
```

Figura 2: Instalar Ngrok

3 Prueba con API test simple

Volver

Ir al dashboard de Coolify (localhost:8000)

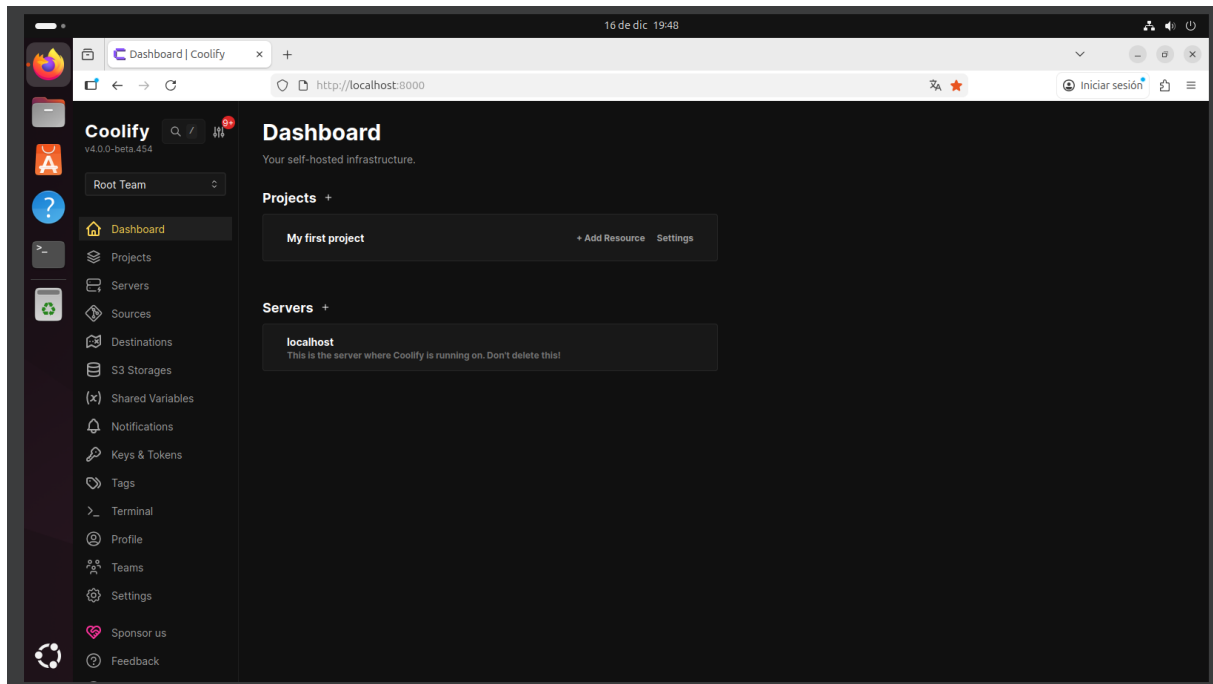


Figura 3: Dashboard

Crear nuevo proyecto, añadir recurso (repositorio público, <https://github.com/1001inchNails/apiTestCicd>)

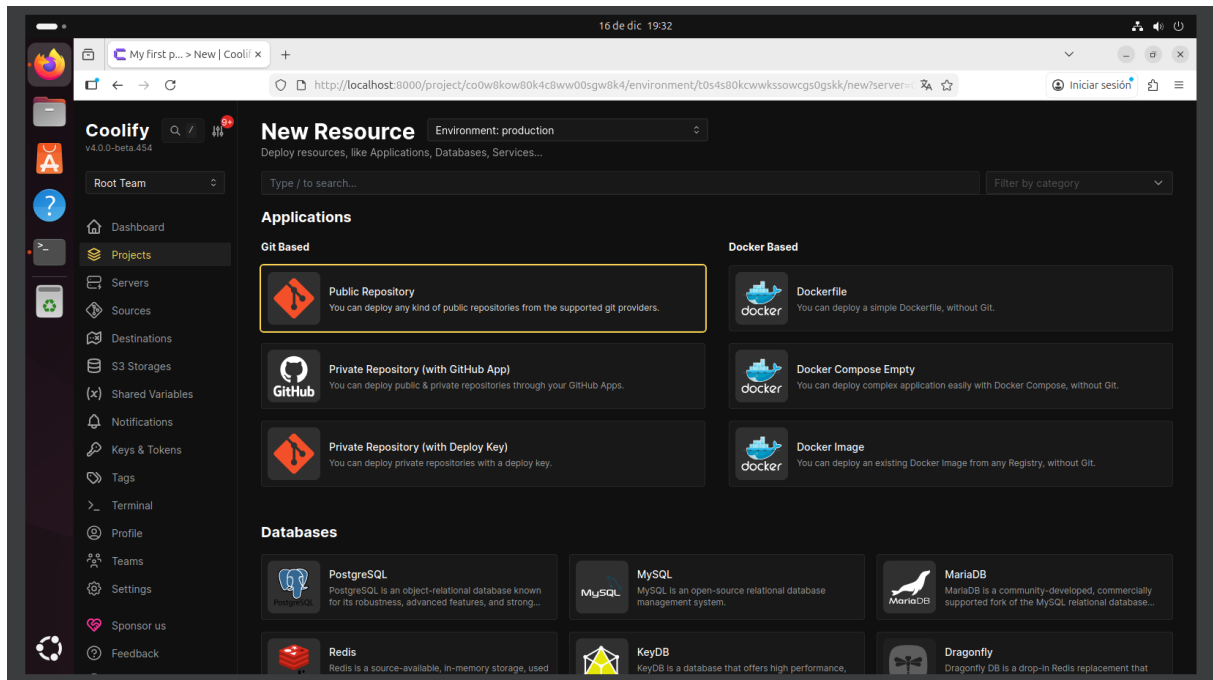


Figura 4: Añadir recurso

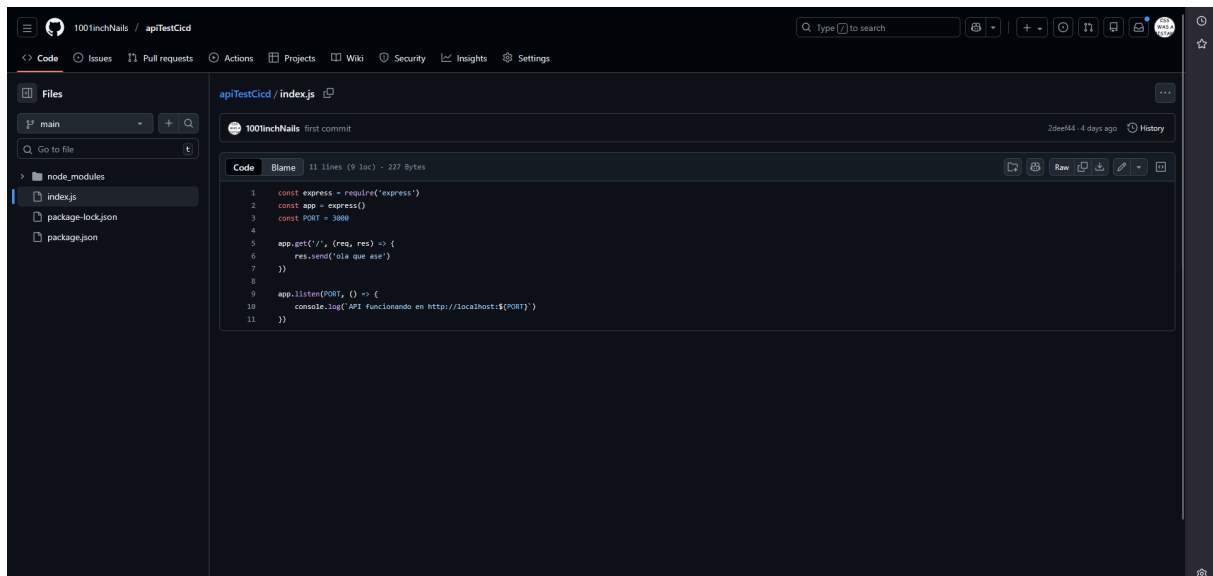


Figura 5: Test API

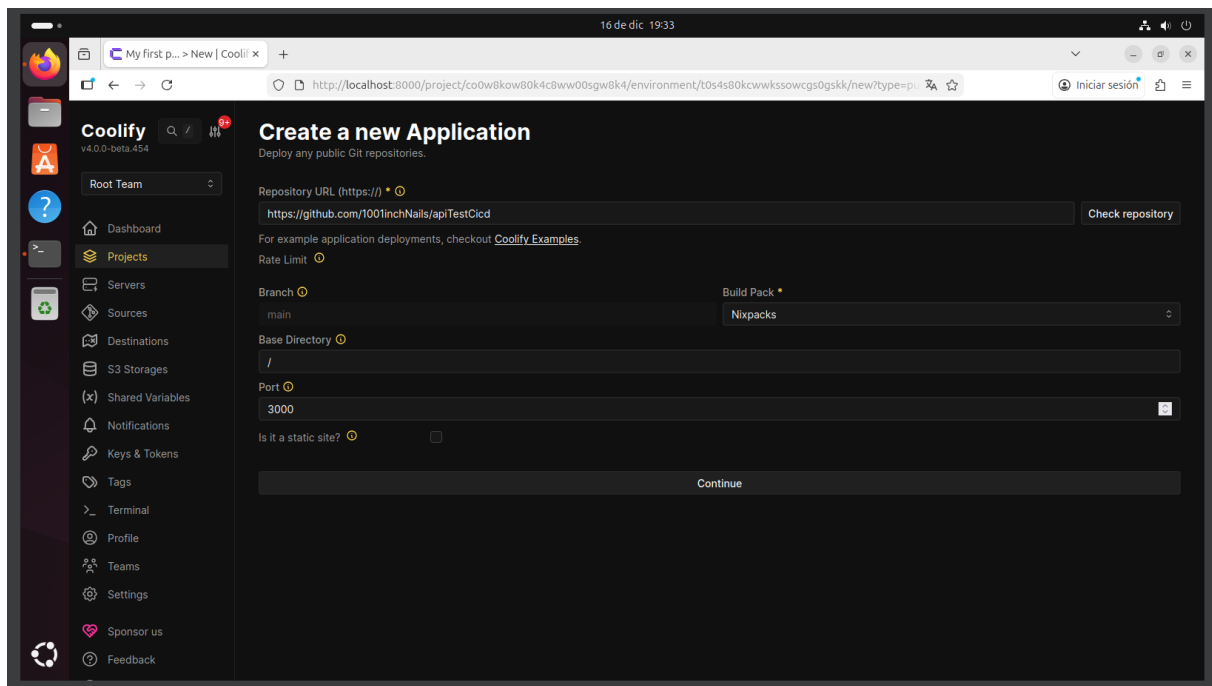


Figura 6: Nixpacks, puerto 3000

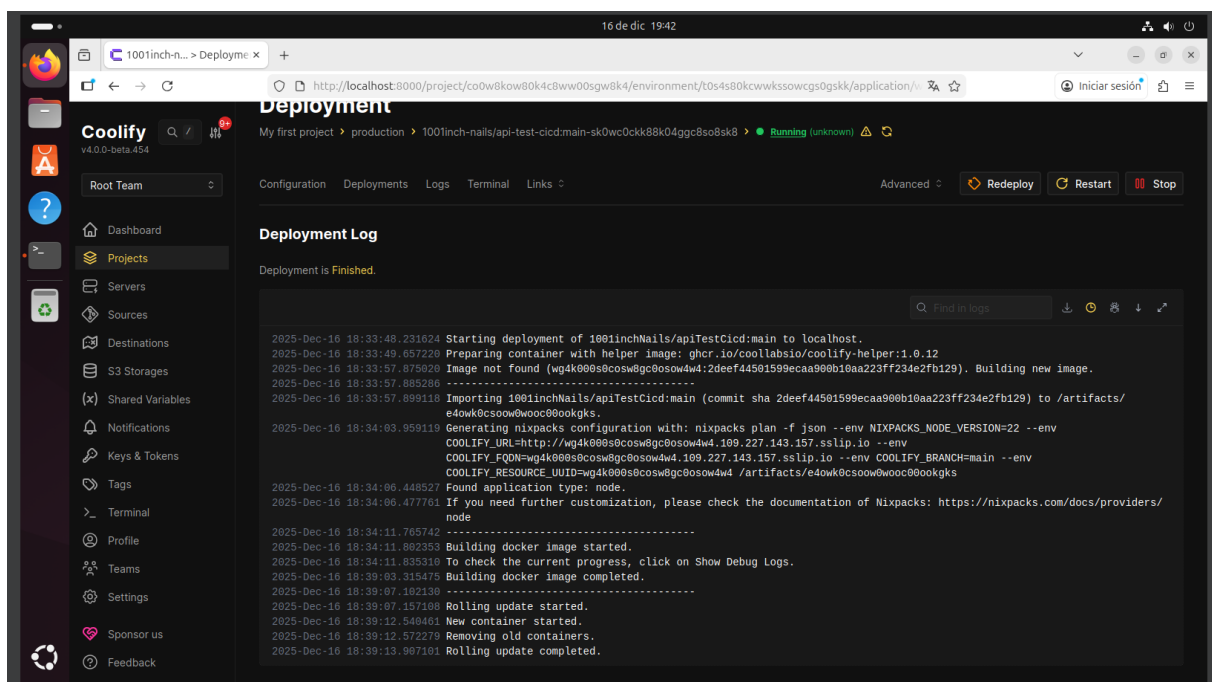


Figura 7: Desplegar la API

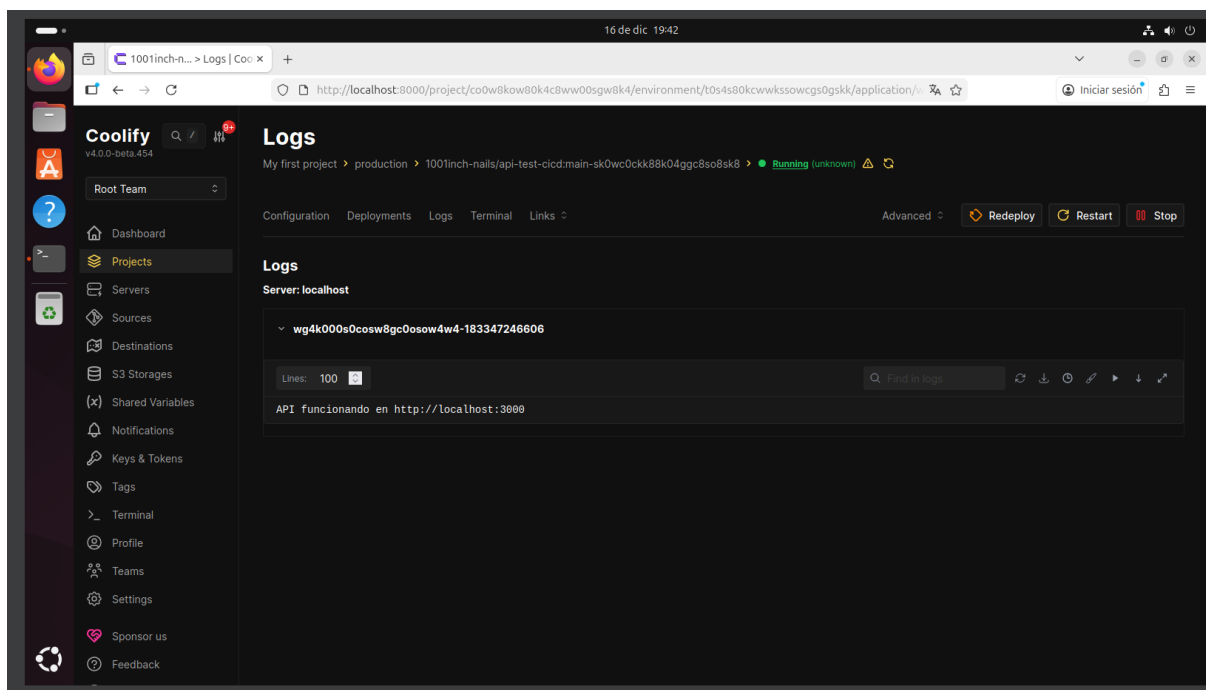


Figura 8: Comprobar logs

Buscamos la ID del contenedor con nuestra API:

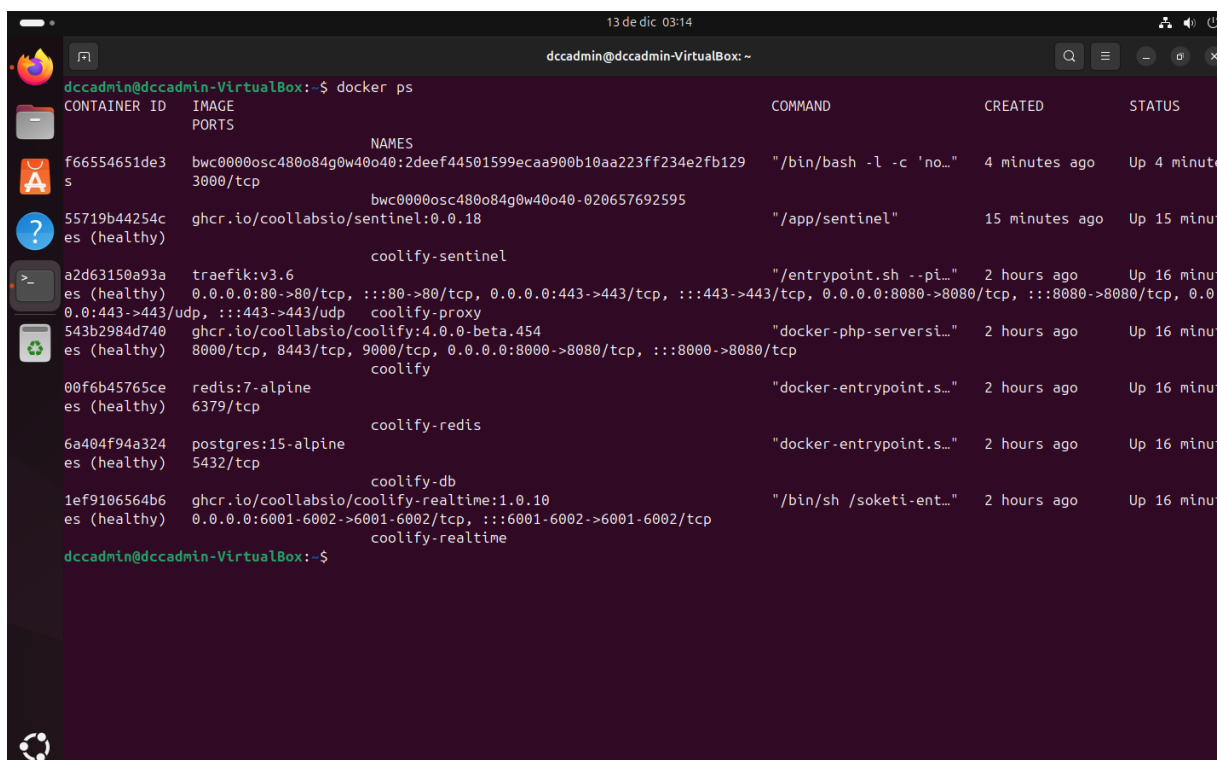


Figura 9: En este caso, el primero

Con esa ID, buscamos la IP del contenedor:

```

1ef9106564b6  ghcr.io/coolabsio/coolify-realtime:1.0.10      "/bin/sh /soketi-ent..." 2 hours ago    Up 16 min
es (healthy)  0.0.0.0:6001-6002->6001-6002/tcp, :::6001-6002->6001-6002/tcp
coolify-realtime
dccadmin@dccadmin-VirtualBox:~$ docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' f66554651de3
10.0.1.8
dccadmin@dccadmin-VirtualBox:~$

```

Figura 10: IP del contenedor con nuestra API

Comprobación en máquina local:

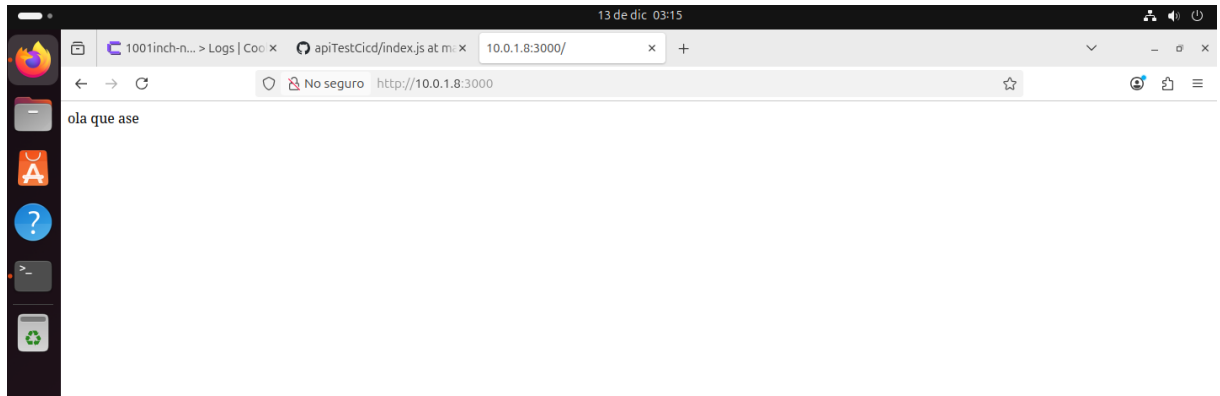


Figura 11: Comprobación

Exponemos con Ngrok:

```

dccadmin@dccadmin-VirtualBox:~$ docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' f66554651de3
10.0.1.8
dccadmin@dccadmin-VirtualBox:~$ ngrok http 10.0.1.8:3000

```

Figura 12: Creación de "túnel" para exponer la API de local a exterior

```

ngrok
Call internal services from your gateway: https://ngrok.com/r/http-request

Session Status      online
Account             danielcc@aulaestudio.es (Plan: Free)
Version             3.34.1
Region              Europe (eu)
Web Interface        http://127.0.0.1:4040
Forwarding            https://magdalene-parotidean-deceptively.ngrok-free.dev -> http://10.0.1.8:3000

Connections
ttl    opn    rt1    rt5    p50    p90
0       0       0.00   0.00   0.00   0.00

```

Figura 13: No cerrar el terminal

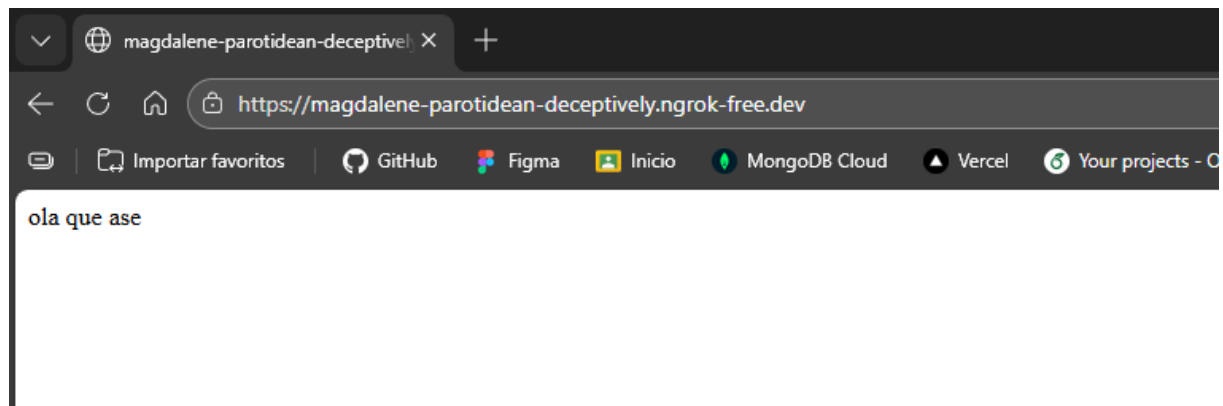


Figura 14: Prueba en maquina host

4 Prueba con API/BBDD

Volver

La práctica se desarrollará desde la máquina host, para ello exponemos el puerto 8000, usado por el dashboard de Coolify:

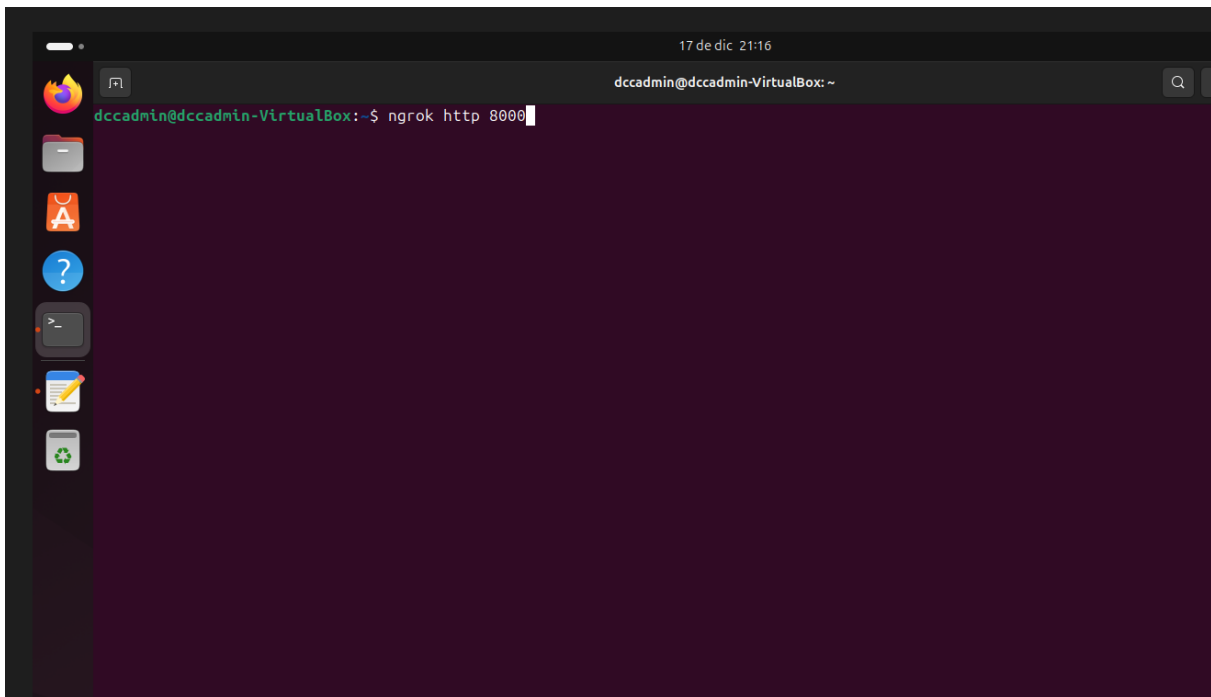


Figura 15: Exponer puerto

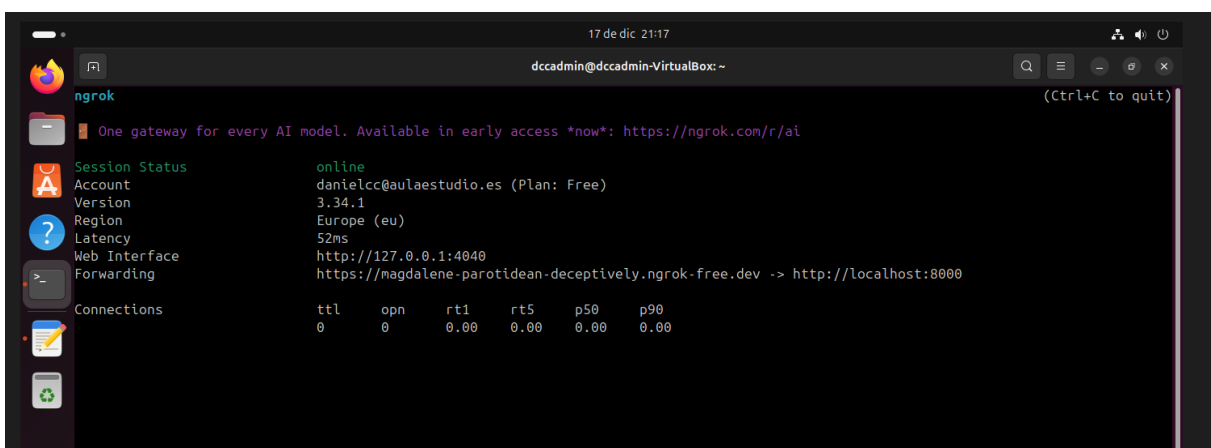


Figura 16: Tunnel

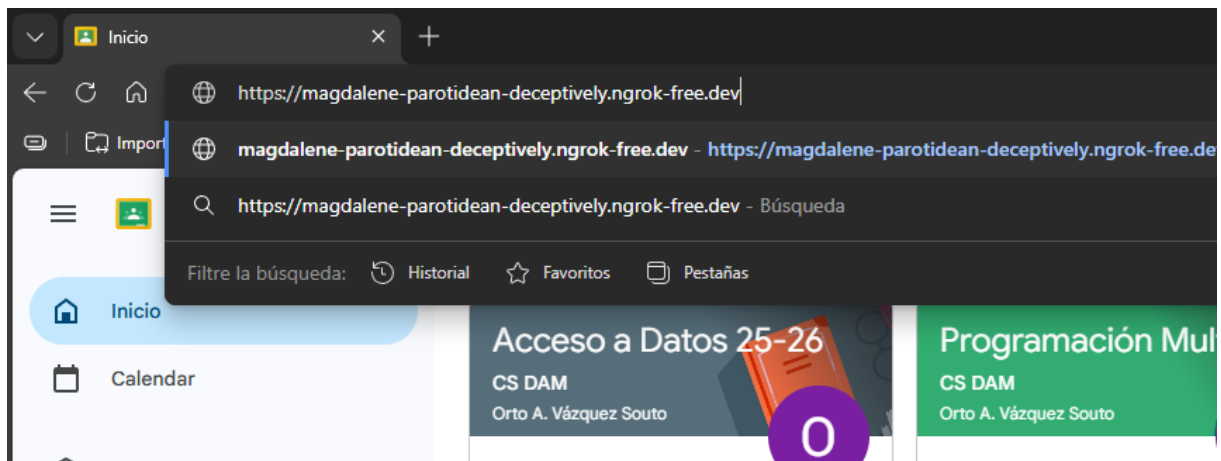


Figura 17: En host

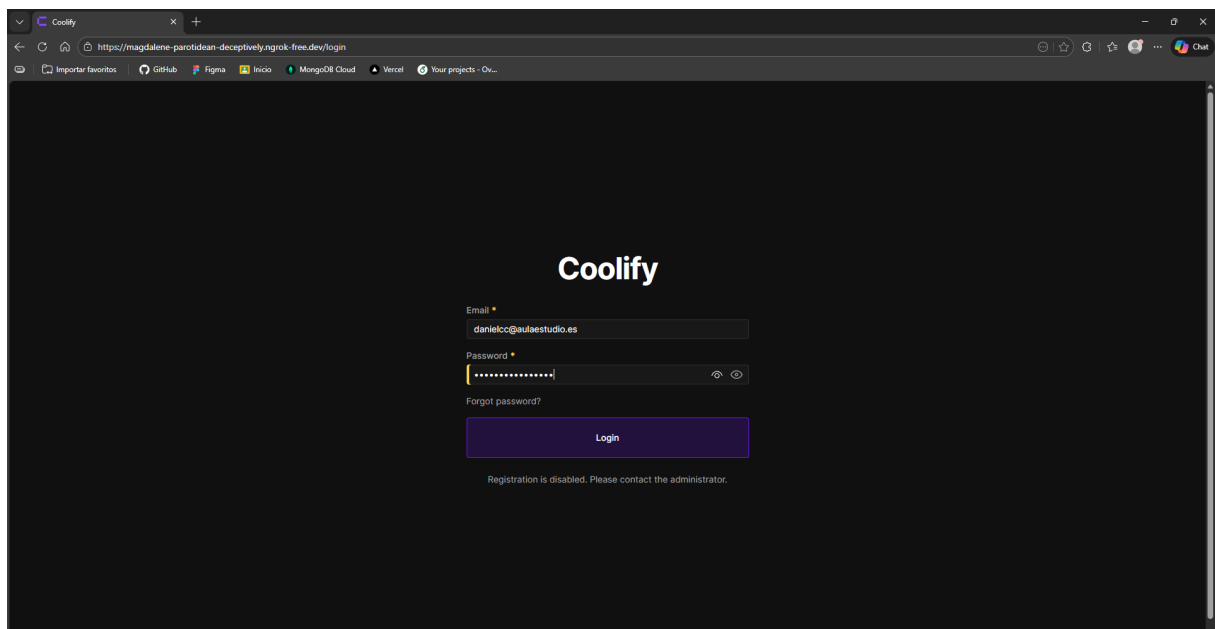


Figura 18: Acceso a dashboard

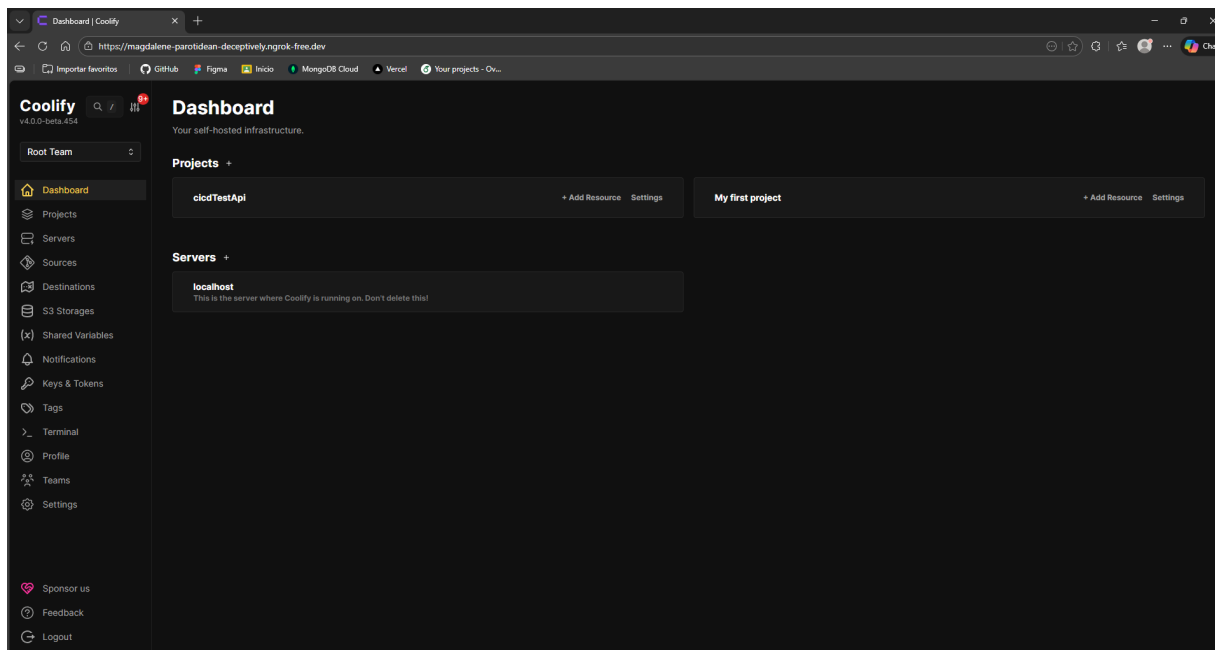


Figura 19: Dashboard de Coolify de VM en máquina host

Creamos un nuevo proyecto:

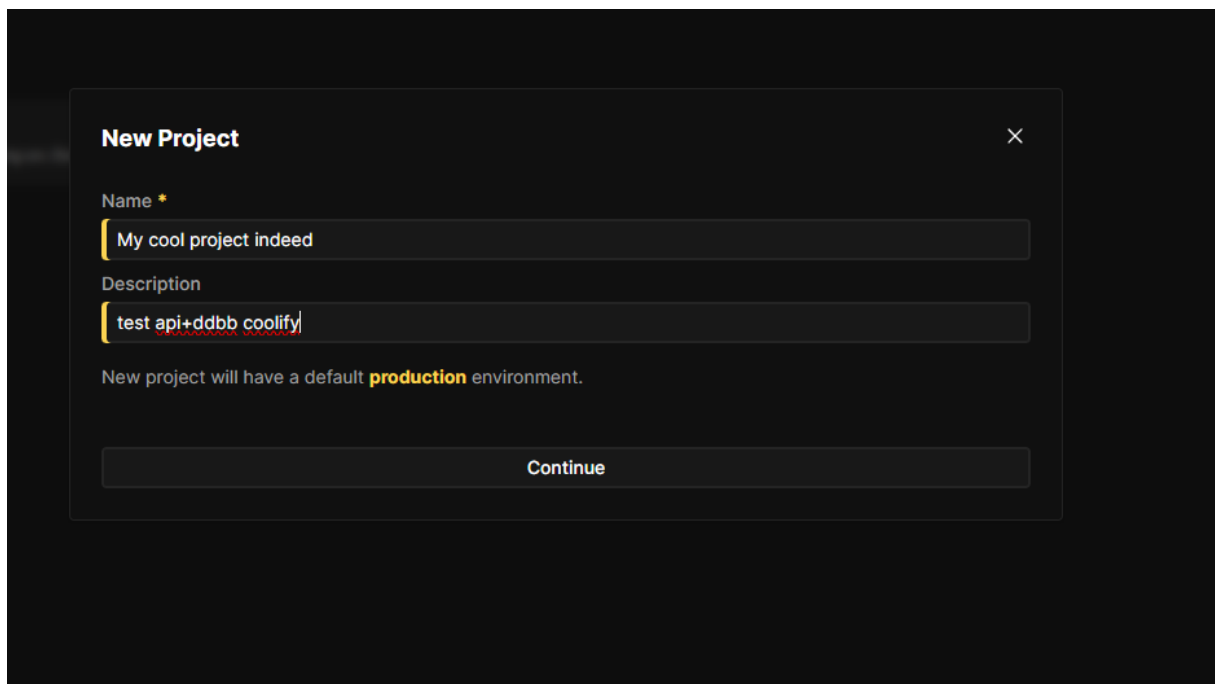


Figura 20: Crear nuevo proyecto

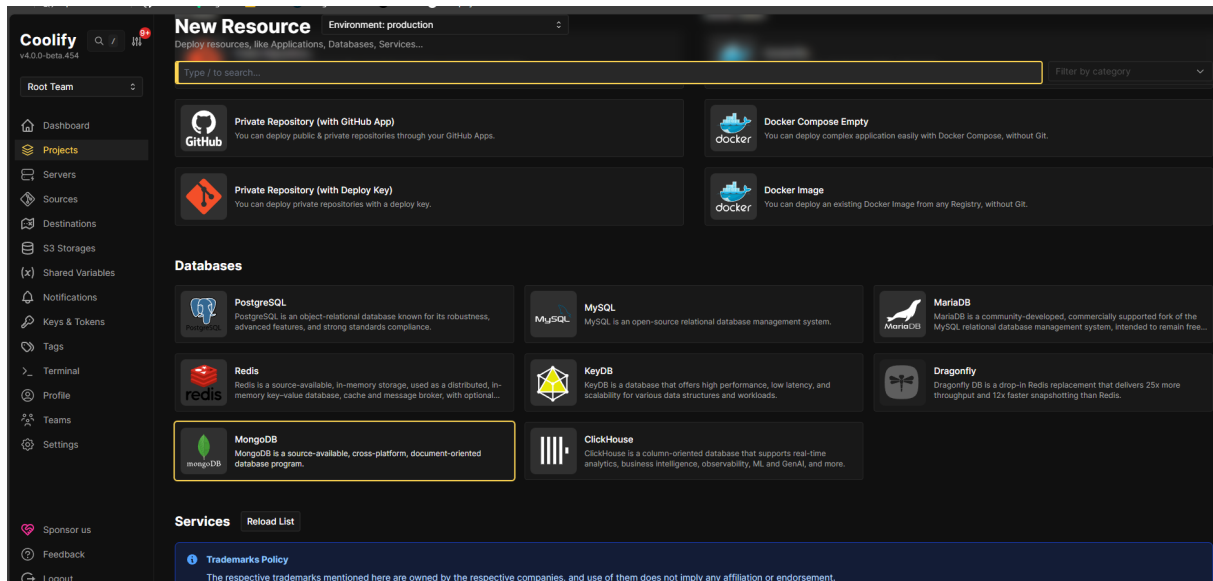


Figura 21: Añadimos recurso: BBDD

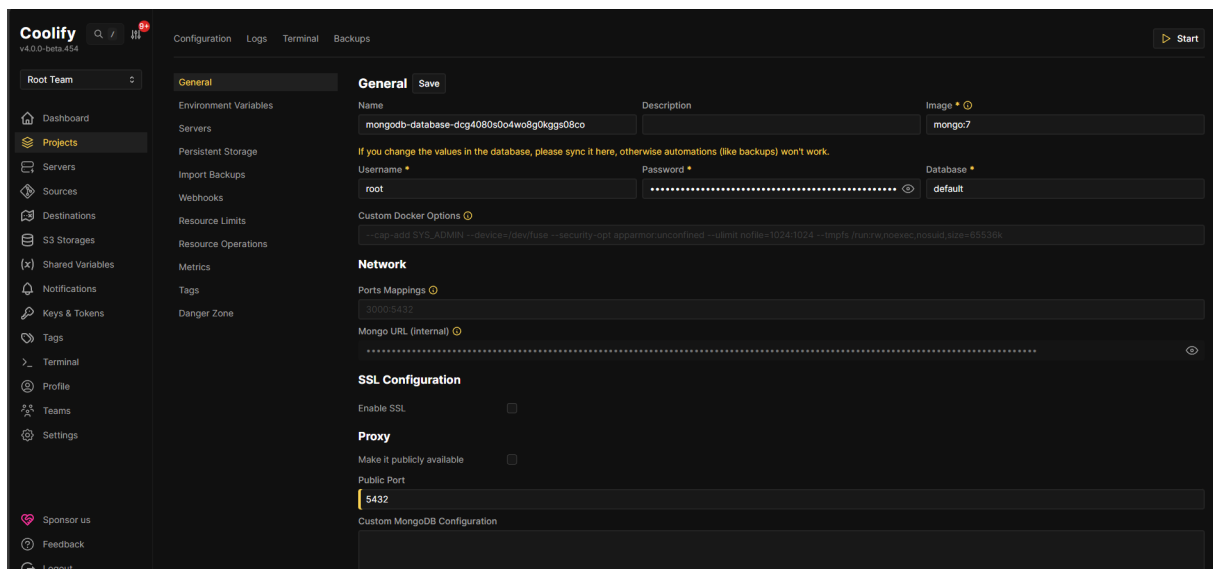


Figura 22: Configurar puertos

Ponerle la contraseña generada en la variable de entorno:

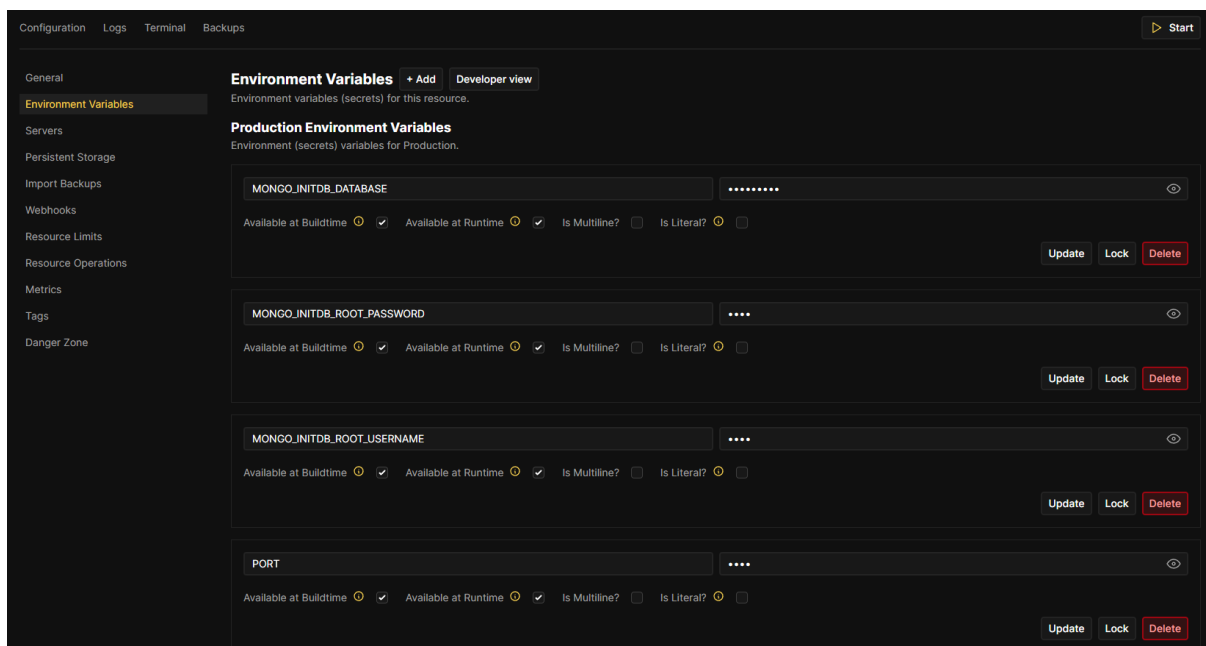


Figura 23: Meter variables de entorno

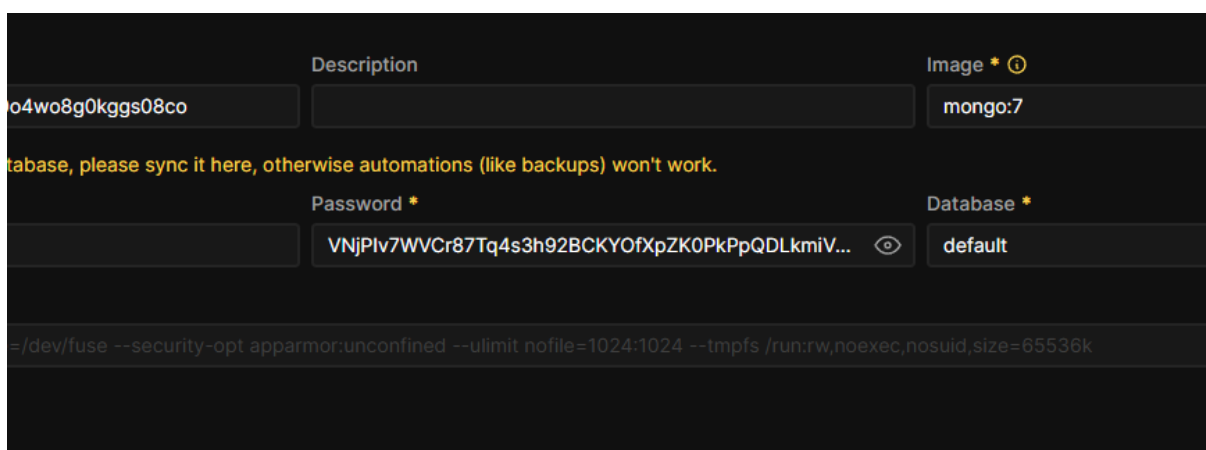


Figura 24: Esta

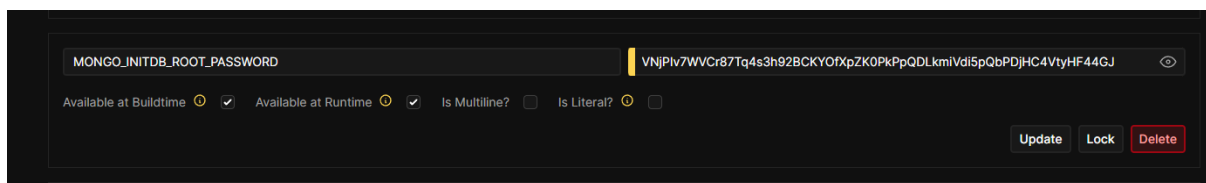
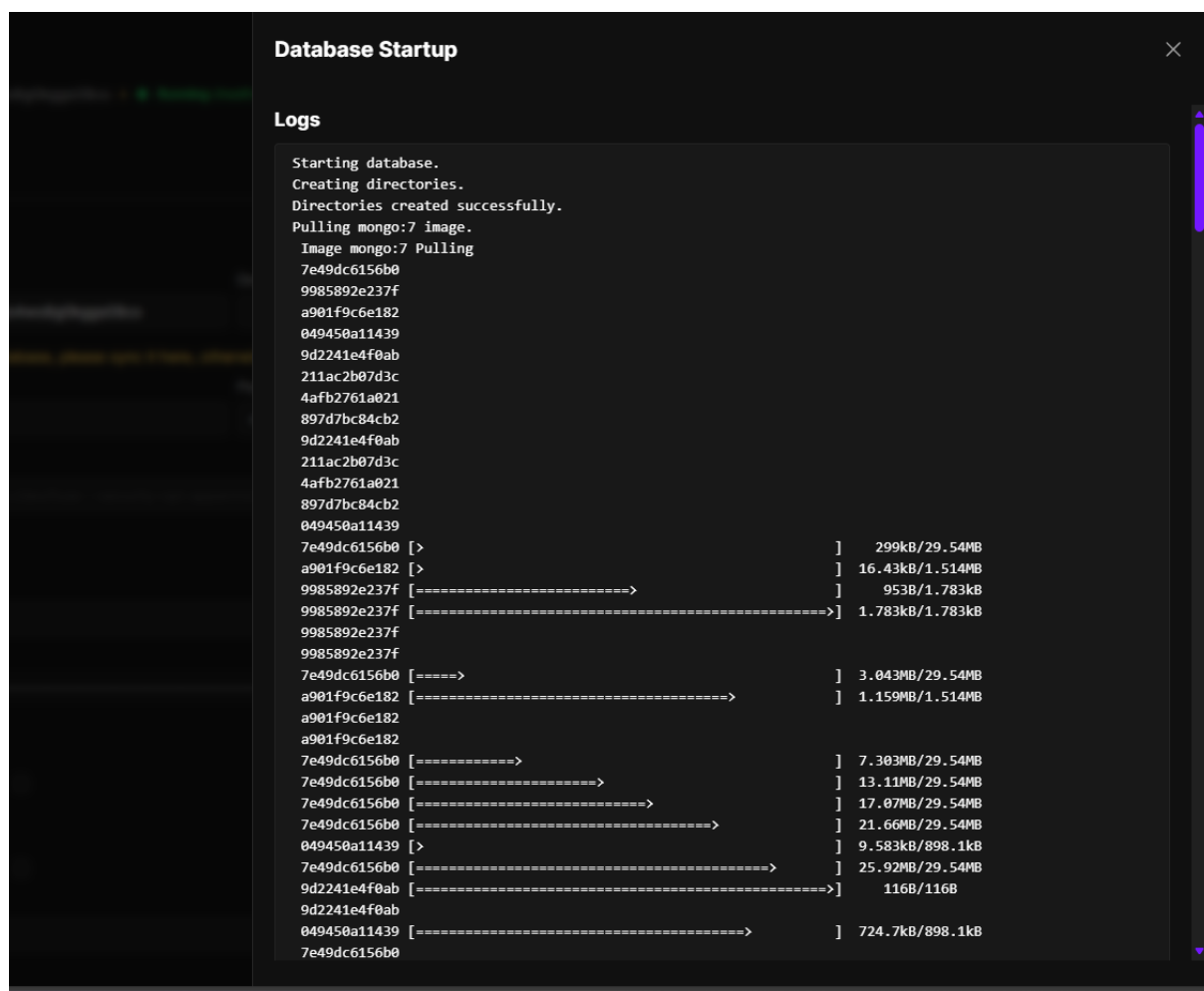


Figura 25: Aquí



```
Database Startup

Logs

Starting database.
Creating directories.
Directories created successfully.
Pulling mongo:7 image.
Image mongo:7 Pulling
7e49dc6156b0
9985892e237f
a901f9c6e182
049450a11439
9d2241e4f0ab
211ac2b07d3c
4afb2761a021
897d7bc84cb2
9d2241e4f0ab
211ac2b07d3c
4afb2761a021
897d7bc84cb2
049450a11439
7e49dc6156b0 [ > ] 299kB/29.54MB
a901f9c6e182 [ > ] 16.43kB/1.514MB
9985892e237f [=====>] 953B/1.783kB
9985892e237f [=====>] 1.783kB/1.783kB
9985892e237f
9985892e237f
7e49dc6156b0 [=====>] 3.043MB/29.54MB
a901f9c6e182 [=====>] 1.159MB/1.514MB
a901f9c6e182
a901f9c6e182
7e49dc6156b0 [=====>] 7.303MB/29.54MB
7e49dc6156b0 [=====>] 13.11MB/29.54MB
7e49dc6156b0 [=====>] 17.07MB/29.54MB
7e49dc6156b0 [=====>] 21.66MB/29.54MB
049450a11439 [ > ] 9.583kB/898.1kB
7e49dc6156b0 [=====>] 25.92MB/29.54MB
9d2241e4f0ab [=====>] 116B/116B
9d2241e4f0ab
049450a11439 [=====>] 724.7kB/898.1kB
7e49dc6156b0
```

Figura 26: Desplegamos

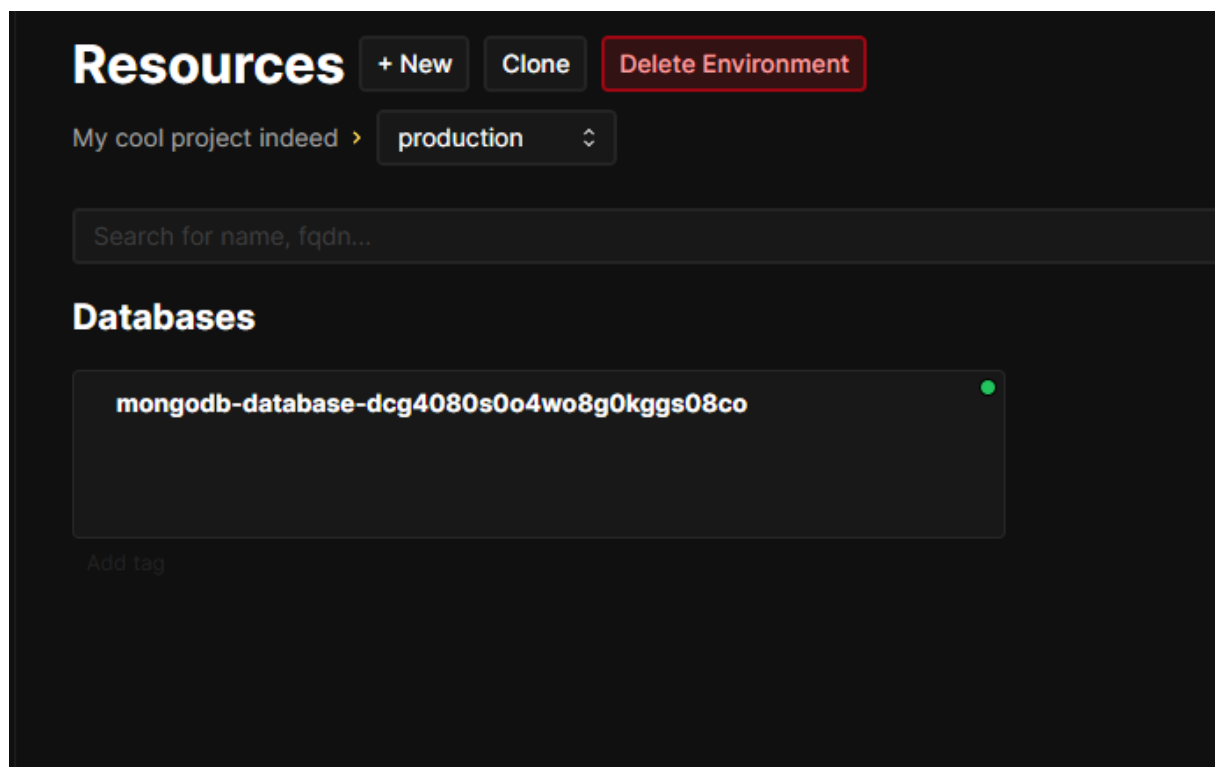


Figura 27: Despliegue correcto

Ahora creamos nuevo recurso, en este caso con la API creada en la práctica anterior (https://github.com/1001inchNails/afond_1-4).

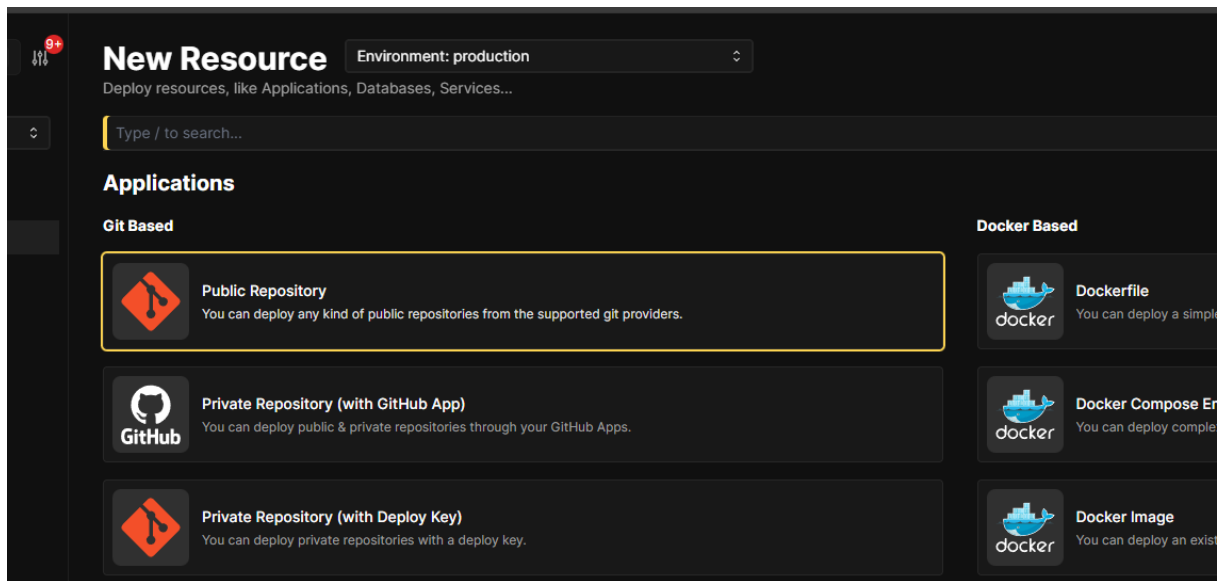


Figura 28: Crear nuevo recurso

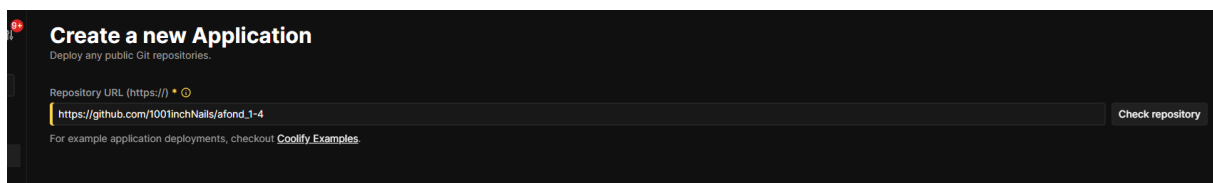


Figura 29: Repositorio público

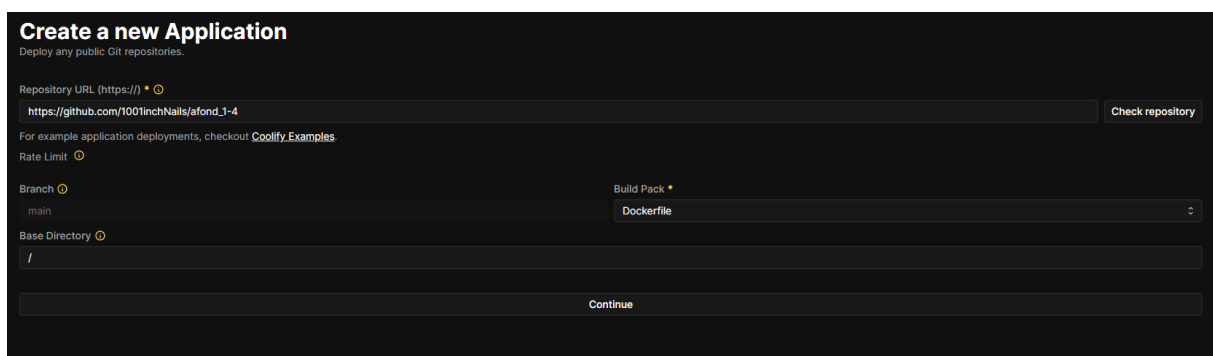


Figura 30: Build: Dockerfile

Environment Variables + Add Developer view

Environment variables (secrets) for this resource.

Sort alphabetically ☐

Use Docker Build Secrets ☐

Production Environment Variables

Environment (secrets) variables for Production.

MONGO_URI [password mask] ☐

Available at Buildtime ☒ Available at Runtime ☒ Is Multiline? ☐ Is Literal? ☐

Update Lock Delete

PORT [....] ☐

Available at Buildtime ☒ Available at Runtime ☒ Is Multiline? ☐ Is Literal? ☐

Update Lock Delete

Preview Deployments Environment Variables

Figura 31: Variables de entorno: meterle el password en MONGO_URI

General Save

General configuration for your application.

Name * 1001inch-nails/afond_1-4:main-vs8w8cg4owgcsw40ccwoco8 Description

Build Pack * Dockerfile

Domains ☐ http://x4skssk0wk4wgcosckwwgcwc.91.117.232.105.sslip.io Generate Domain

Direction * ☐ Allow www & non-www. Set Direction

Docker Registry ☐

Docker Image ☐ Empty means it won't push the image to a docker registry. Docker Image Tag ☐ Empty means only push commit sha tag.

Build

Base Directory ☐ / Dockerfile Location ☐ /Dockerfile Docker Build Stage Target ☐

Custom Docker Options ☐ --cap-add SYS_ADMIN --device=/dev/fuse --security-opt apparmor:unconfined --ulimit nofile=1024:1024 --tmpfs /run,runw,nosuid,size=65536k --hostname=myapp

Use a Build Server? ☐

Figura 32: Definir carpeta raiz, y ponerle el mismo puerto que el definido en el código de la API

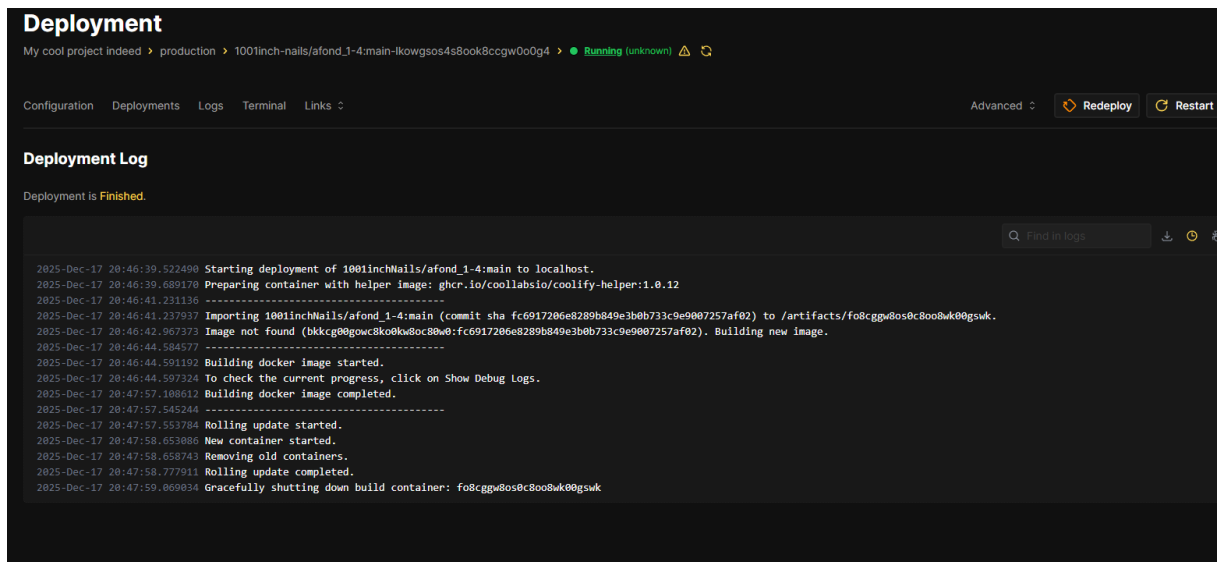


Figura 33: Guardar y desplegar

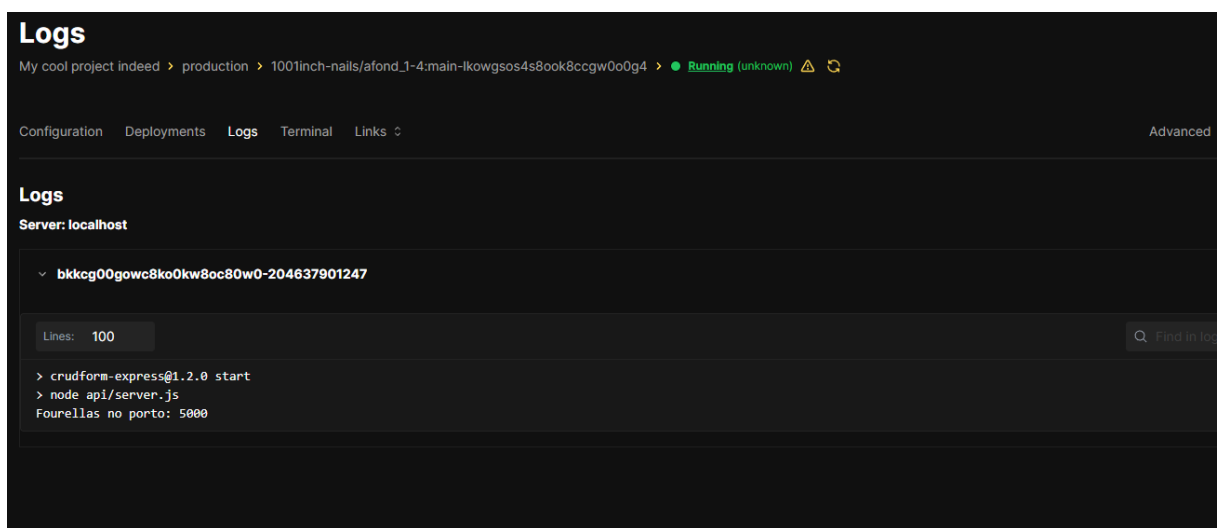
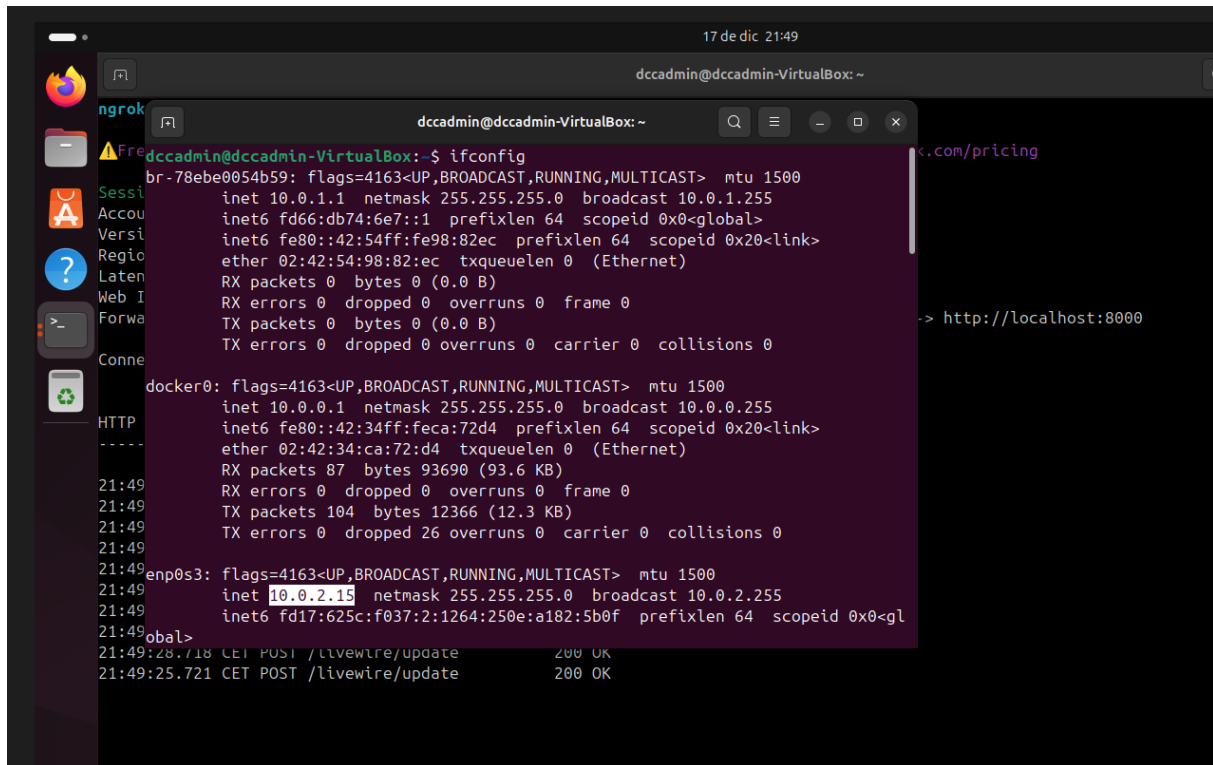


Figura 34: Log

Ahora buscamos la IP de la VM:



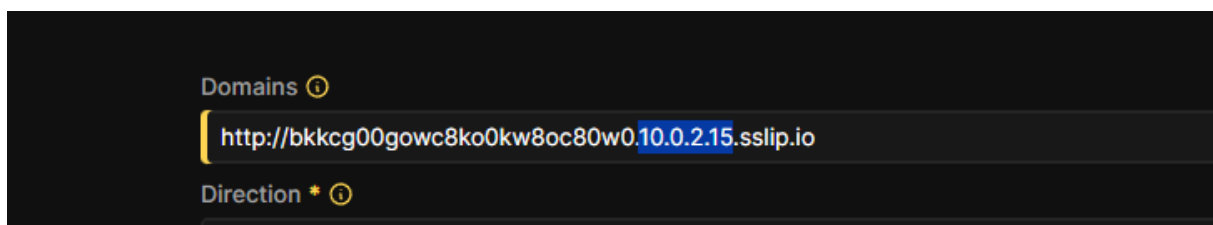
```
17 de dic 21:49
dccadmin@dccadmin-VirtualBox: ~
$ ifconfig
br-78be0054b59: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.1.1 netmask 255.255.255.0 broadcast 10.0.1.255
    inet6 fd66:db74:6e7::1 prefixlen 64 scopeid 0x0<global>
    inet6 fe80::42:54ff:fe98:82ec prefixlen 64 scopeid 0x20<link>
    ether 02:42:54:98:82:ec txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

docker0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.1 netmask 255.255.255.0 broadcast 10.0.0.255
    inet6 fe80::42:34ff:feca:72d4 prefixlen 64 scopeid 0x20<link>
    ether 02:42:34:ca:72:d4 txqueuelen 0 (Ethernet)
    RX packets 87 bytes 93690 (93.6 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 104 bytes 12366 (12.3 KB)
    TX errors 0 dropped 26 overruns 0 carrier 0 collisions 0

enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
    inet6 fd17:625c:f037:2:1264:250e:a182:5b0f prefixlen 64 scopeid 0x0<gl
    global>
21:49:28.718 CET POST /livewire/update 200 OK
21:49:25.721 CET POST /livewire/update 200 OK
```

Figura 35: Interfaces de red (enp0s3)

Metemos esa IP en el dominio que nos proporciona Coolify:



```
Domains ⓘ
http://bkkcg00gowc8ko0kw8oc80w0.10.0.2.15.sslip.io
Direction * ⓘ
```

Figura 36: sslip

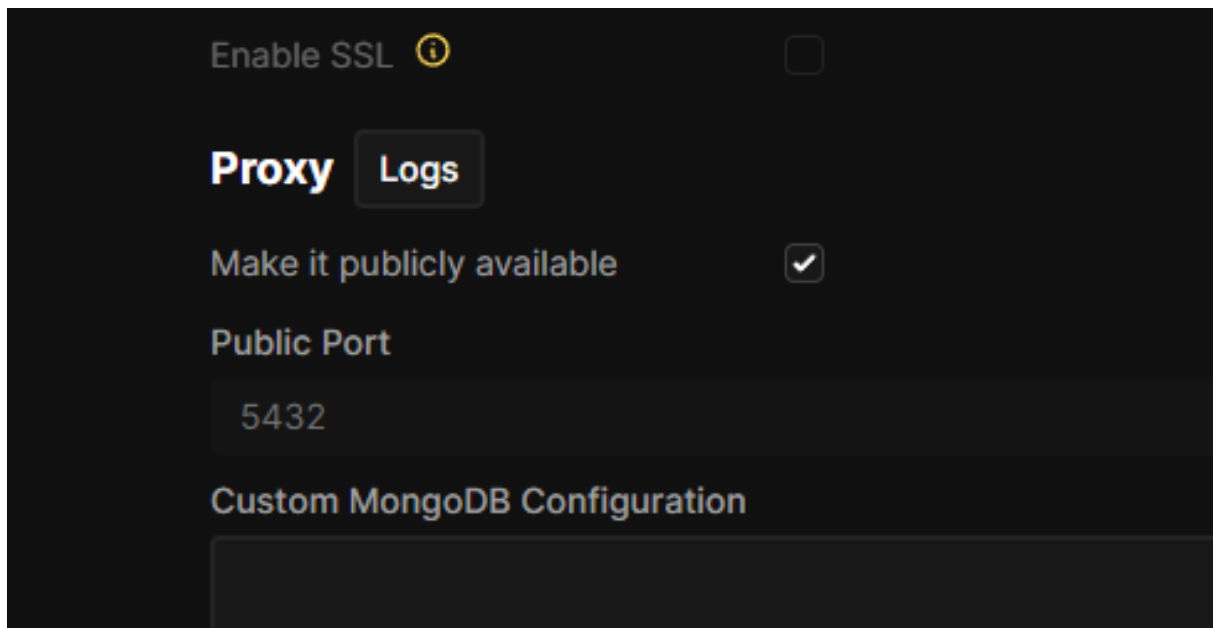


Figura 37: Hacerlo disponible a público

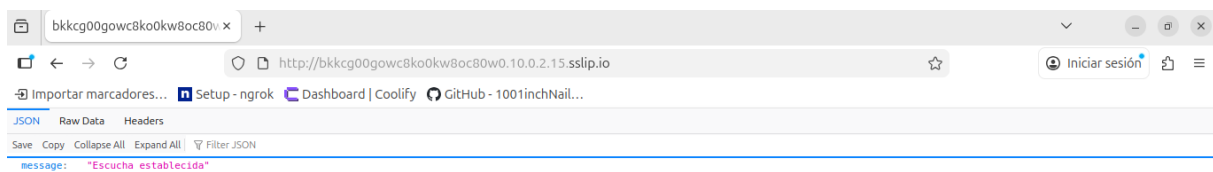


Figura 38: Correcto funcionamiento: VM

En este momento se descubrió un problema, en la VM el enlace cargaba correctamente pero no en la máquina host. Esto se debía a que la configuración de red de la VM no estaba en modo puente.

Después de cambiarlo, se procedió a volver a ejecutar `ifconfig` para buscar la IP nueva, meterla en el dominio y, esta vez, cargaba correctamente en host:

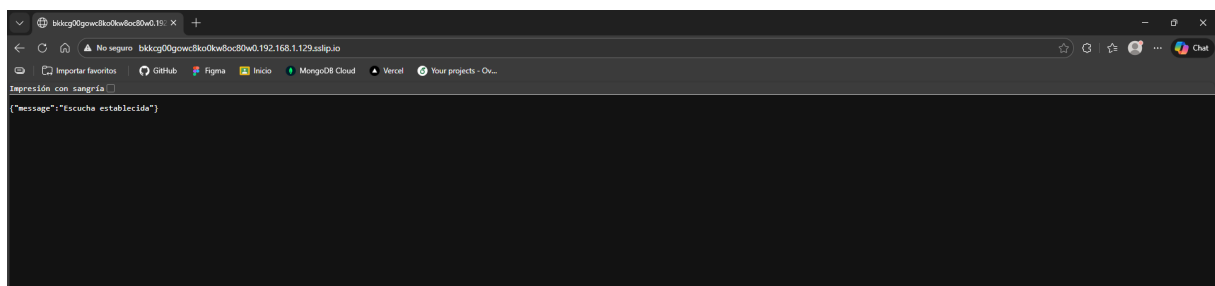


Figura 39: MISSION ACCOMPLISHED

5 Abrir tunel fuera de la red

Volver

Para abrir un tunel con ngrok y poder ver la API fuera de nuestra red, necesitamos usar este comando, usando el puerto 80 (por defecto de ngrok) y usar el dominio del webhook proporcionado por coolify:

```
ngrok http 80 --host-header=l0wcs0g0wcscg0occc8gg8cw.192.168.0.66.sslip.io
```

Con el enlace resultante podemos ver la API en cualquier dispositivo.

6 Auto-desplegar con webhook de Coolify

Volver

Para que la aplicación se auto despliegue cada vez que se hace un push al repositorio de github de la API.

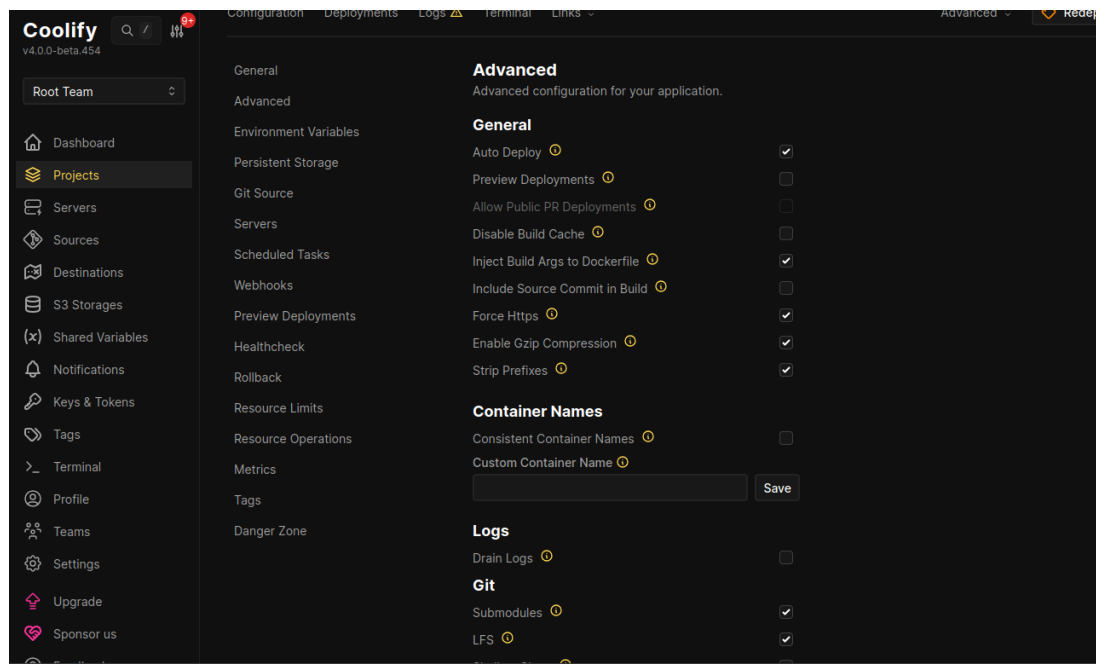


Figura 40: Activar auto deploy en la configuración avanzada del proyecto (Coolify dashboard)

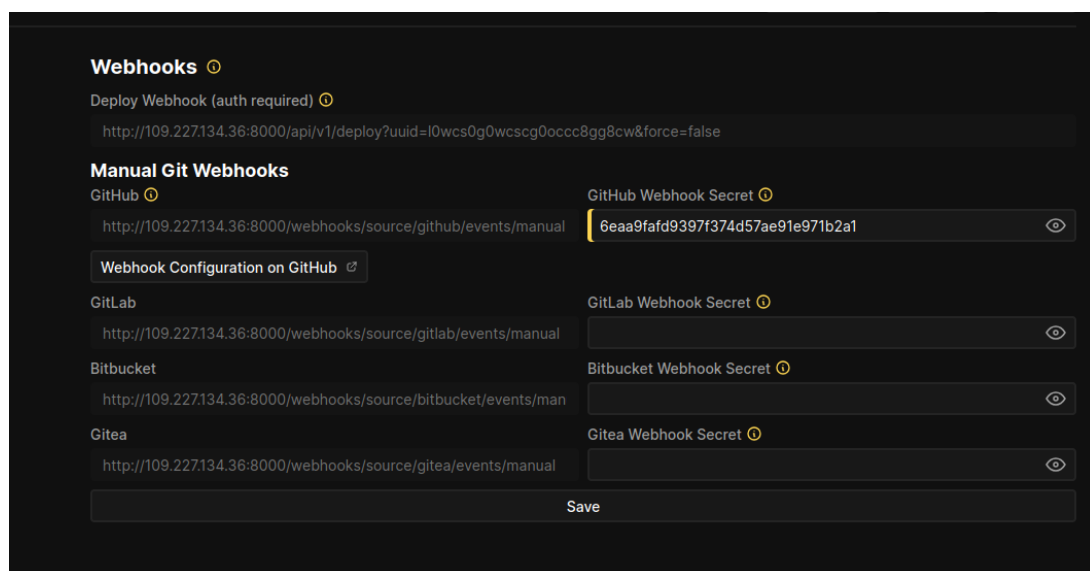


Figura 41: Ir a la pestaña de Webhooks, crear un secret y ponerselo.

Ir a la pestaña Settings del repositorio de nuestra API, clickea Webhooks.

En payload URL introduce el Manual Git Webhook de Github, pero debes sustituir el dominio anterior a /webhooks por el dominio que nos da ngrok después de introducir el comando `ngrok http 8000`.

Ponemos el mismo secret que pusimos en la configuración de Coolify

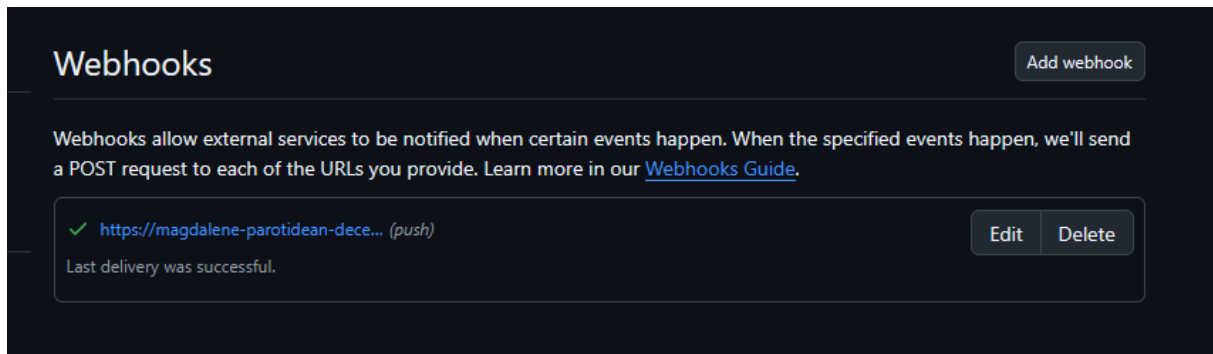


Figura 42: Comprobamos que haga el ping correctamente

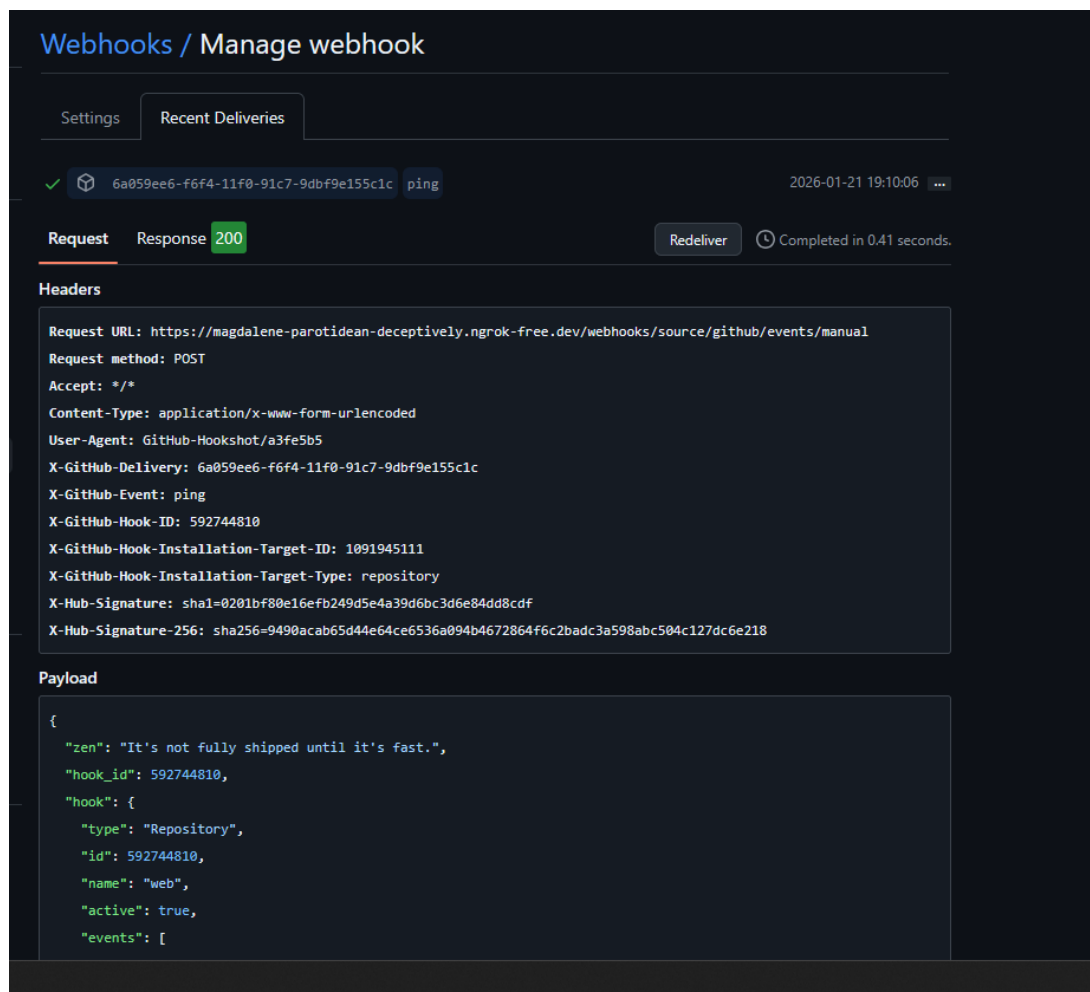


Figura 43: Success!

Ahora comprobamos con el webhook de Coolify <http://10wcs0g0wscsg0occc8gg8cw.192.168.0.68.sslip.io> el estado actual de la API

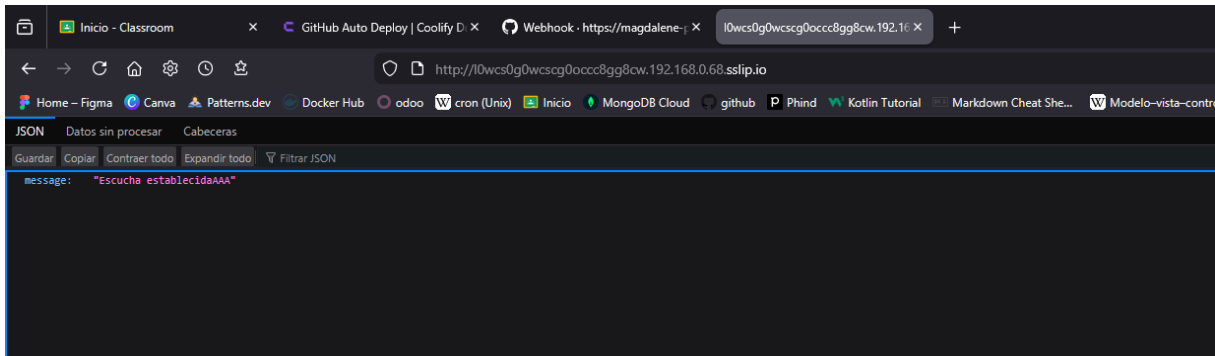


Figura 44: Estado actual

Hacemos un cambio menor a nuestra API.

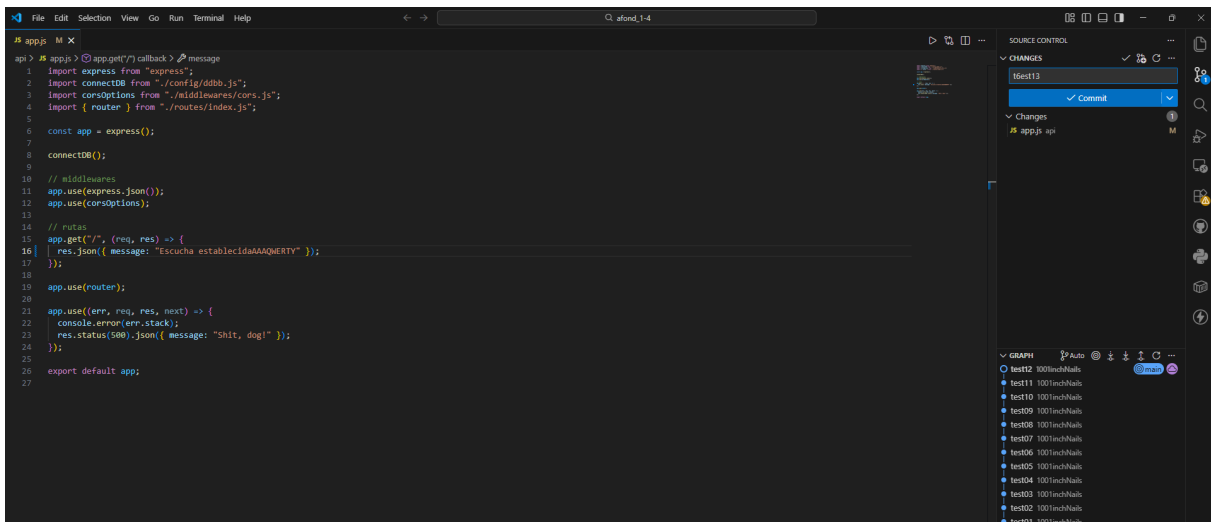


Figura 45: Cambio menor 'QWERTY'

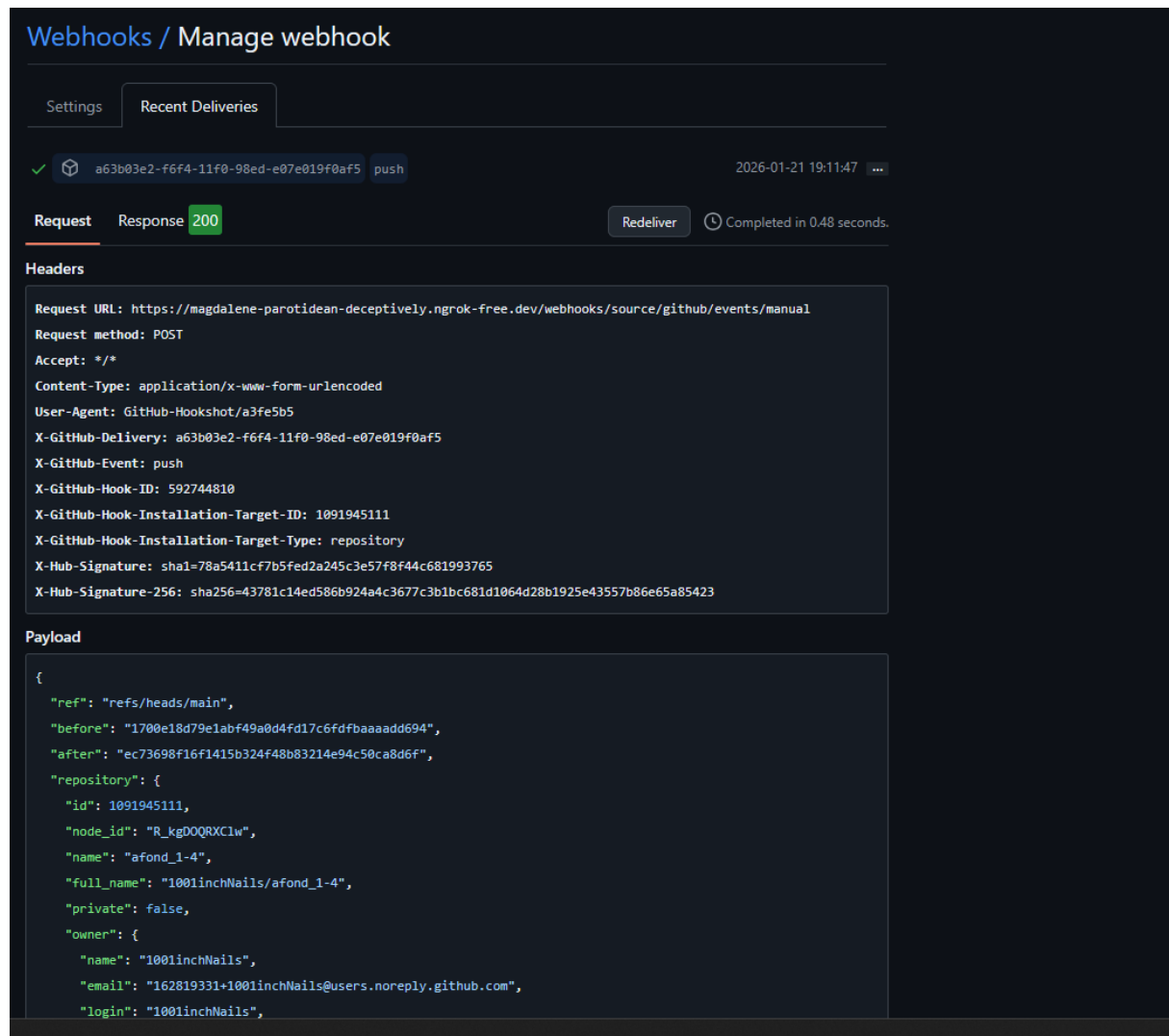


Figura 46: Comprobamos en Github que el push se haya detectado correctamente

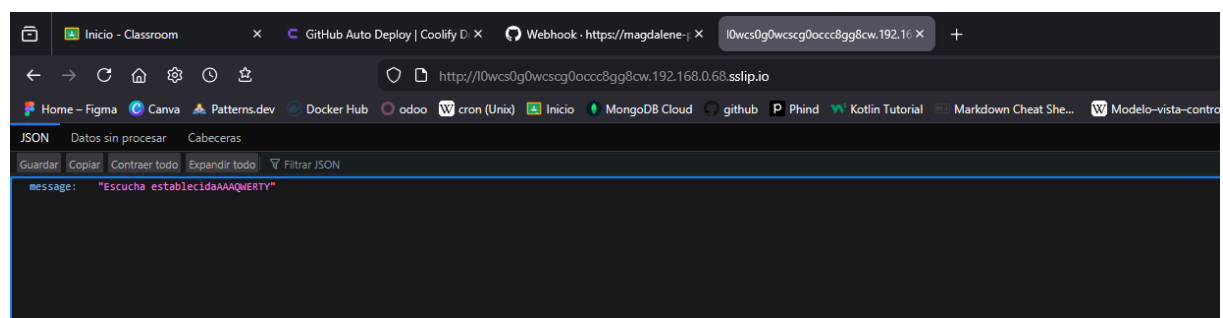


Figura 47: Volvemos al navegador y F5 para comprobar que el auto deploy se ha ejecutado correctamente

Problemas encontrados:

La dirección IP de la máquina virtual cambia entre sesiones, de manera que hay que comprobarla y poner la correcta en el webhook proporcionado por Coolify.

7 Info

Volver

Repositorio

Documentacion https://github.com/1001inchNails/afond_1_5

API de prueba https://github.com/1001inchNails/afond_1-4

Referencia Auto-Deploy <https://coolify.io/docs/applications/ci-cd/github/auto-deploy>

Comandos:

- Instalar Coolify: `curl -fsSL https://cdn.coollabs.io/coolify/install.sh` `sudo bash`

- Instalar Ngrok:

```
curl -sSL https://ngrok-agent.s3.amazonaws.com/ngrok.asc  
sudo tee /etc/apt/trusted.gpg.d/ngrok.asc >/dev/null  
&& echo "deb https://ngrok-agent.s3.amazonaws.com bookworm main"  
| sudo tee /etc/apt/sources.list.d/ngrok.list  
&& sudo apt update  
&& sudo apt install ngrok
```

- Añadir authtoken: `ngrok config add-authtoken <authtoken>`
- Información contenedores Docker: `docker ps`
- IP de contenedor: `docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}`
- Exponer API test con Ngrok: `ngrok http <ip>:<puerto>`
- Exponer puerto con Ngrok: `ngrok http <puerto>`
- Interfaces de red: `ifconfig`